

PROJET DE FIN D'ÉTUDES

MarketCraft

Marketplace artisanale enrichie par l'Intelligence Artificielle

Présenté par :

Antoine Brigouleix

Étudiant en Master 1 — Concepteur Développeur d'Applications

Encadrant :

M. BAKIR Djamel

Tuteur pédagogique

Établissement : INSTA CFA

Année universitaire : 2025 – 2026

Table des matières

I. Présentation.....	4
A. Présentation du projet MarketCraft.....	4
B. Mon rôle dans le projet.....	4
C. Remerciements.....	4
II. Cahier des charges.....	5
A. Contexte.....	5
B. Objectifs.....	5
C. Utilisateurs cibles.....	5
D. Besoins du projet.....	5
E. Module d'Intelligence Artificielle.....	5
F. Temps de réalisation.....	6
III. Conception.....	7
A. Diagramme de contexte.....	7
B. Diagramme de packages.....	7
C. Diagramme de cas d'utilisation.....	7
D. Scénarios.....	11
1. Rechercher un produit avec l'IA et commander.....	11
2. Créer une boutique et publier un produit (vendeur).....	13
3. Déposer un avis vérifié.....	15
E. Modèle conceptuel des données (MCD).....	16
F. Diagramme de classes.....	18
G. Modèle physique de données (MPD).....	20
H. Diagrammes d'activité.....	21
IV. Tests.....	27
A. Objectifs des tests.....	27
B. Types de tests réalisés.....	27
C. Exemple de scénarios de test.....	27
D. Limites et améliorations possibles.....	27
V. Développement.....	29
A. Technologies choisies.....	29
B. Environnement de travail.....	29
C. Architecture MVC / Installation.....	29
D. Backend.....	30
1. Modèles.....	30
2. Routes / API.....	31
3. Contrôleurs.....	31
4. Sécurité / Authentification.....	31
E. Frontend (React).....	34
F. Application mobile (React Native).....	34
G. Module IA — recherche et recommandation.....	34

H. Déploiement.....	35
VI. Perspectives.....	36
VII. Conclusion.....	36
VIII. Webographie.....	36
IX. Annexes.....	37
Annexe A — Structure de la base de données.....	37
Annexe B — Exemple de payload JWT.....	37
Annexe C — Exemple de requête API (Axios).....	37
Annexe D — Module IA : réponse JSON.....	37
Annexe E — Maquettes de l'interface.....	38

I. Présentation

A. Présentation du projet MarketCraft

MarketCraft est une application web et mobile de type marketplace dédiée à l'artisanat. Le projet est né d'un double constat : d'une part, de nombreux artisans peinent à vendre leurs créations en ligne faute de boutique adaptée et d'une visibilité suffisante ; d'autre part, les acheteurs ont du mal à découvrir des produits faits main correspondant réellement à leurs envies, noyés dans des catalogues immenses et des moteurs de recherche rigides.

L'objectif de MarketCraft est d'offrir une plateforme centralisée où chaque artisan peut créer sa boutique, publier ses produits et gérer ses ventes, tandis que l'acheteur navigue, recherche, commande et paie en toute simplicité. La valeur ajoutée du projet réside dans son module d'Intelligence Artificielle : une recherche en langage naturel qui interprète l'intention de l'acheteur pour lui recommander les produits les plus pertinents du catalogue.

B. Mon rôle dans le projet

MarketCraft a été réalisé en binôme dans le cadre du projet de 4^e année. En tant que développeur full-stack, j'ai participé à l'ensemble du cycle de vie du projet, de l'analyse des besoins jusqu'au développement et aux tests. J'ai plus particulièrement pris en charge :

- La conception du modèle de données (MCD, MLD, MPD) et des diagrammes UML.
- Le développement du backend en PHP selon une architecture MVC, et de l'API REST sécurisée par JWT.
- L'intégration du module IA de recherche / recommandation via l'API Claude (Anthropic).
- La contribution au frontend React et à l'application mobile React Native.

Ce travail en équipe m'a permis de mettre en pratique les méthodologies agiles, la gestion de versions avec Git, et la répartition des responsabilités au sein d'un binôme.

C. Remerciements

Je remercie l'Institut National Supérieur des Technologies Avancées (INSTA CFA) pour la qualité de la formation Concepteur Développeur d'Applications, ainsi que l'ensemble de mes professeurs. Je remercie tout particulièrement M. BAKIR Djamel, notre tuteur pédagogique, pour son accompagnement, sa disponibilité et ses conseils tout au long de ce projet.

Je remercie également mon binôme pour son investissement, ainsi que mes proches pour leur soutien constant durant cette année.

II. Cahier des charges

A. Contexte

Le commerce en ligne de produits artisanaux est en pleine croissance, mais reste dominé par de grandes plateformes généralistes peu adaptées aux créateurs indépendants. MarketCraft se donne pour mission de valoriser l'artisanat en offrant un outil de vente clé en main, accessible depuis le web comme depuis un smartphone, et en facilitant la découverte des produits grâce à l'Intelligence Artificielle.

B. Objectifs

Les objectifs de MarketCraft sont multiples :

- Offrir aux artisans une interface pour créer leur boutique et gérer leurs produits (CRUD complet).
- Permettre aux acheteurs de parcourir, rechercher, commander et payer des produits artisanaux.
- Intégrer une recherche intelligente en langage naturel pour améliorer la découverte produit.
- Proposer une version mobile native offrant les mêmes fonctionnalités que le web.
- Garantir la sécurité (authentification JWT, hachage des mots de passe, requêtes préparées).

C. Utilisateurs cibles

L'application cible trois profils principaux :

- Les acheteurs : particuliers à la recherche de produits faits main, uniques ou personnalisés.
- Les vendeurs (artisans) : créateurs souhaitant vendre directement leurs réalisations en ligne.
- Les administrateurs : modérateurs de la plateforme, garants de la qualité et de la confiance.

D. Besoins du projet

Le besoin principal est de disposer d'une marketplace complète permettant de gérer le cycle de vente de bout en bout. Il doit notamment être possible de :

- S'inscrire et se connecter de façon sécurisée, avec gestion des rôles (acheteur / vendeur / admin).
- Créer une boutique, y publier des produits avec images, prix, stock et tags.
- Rechercher des produits par mots-clés ou en langage naturel, filtrer par catégorie et par prix.
- Gérer un panier, passer une commande, suivre son statut et déposer un avis après achat.
- Suivre l'activité via des tableaux de bord (vendeur et acheteur).

E. Module d'Intelligence Artificielle

Conformément au cahier des charges, le projet intègre un seul module IA : l'Option C — recommandation et pertinence personnalisée des produits. Concrètement, l'acheteur formule sa recherche en français courant (par exemple « un cadeau fait main en bois pour moins de 50 € »). La requête est envoyée à l'API Claude, qui en extrait des mots-clés, des catégories et une fourchette de prix sous forme de JSON structuré. Le backend interroge alors le catalogue par similarité et renvoie les produits les plus pertinents, accompagnés d'un message convivial. Un mécanisme de repli (fallback) garantit un résultat même si l'IA est indisponible.

F. Temps de réalisation

La réalisation s'est étalée sur six semaines, structurée selon les étapes suivantes :

Période	Activités
Semaine 1	Analyse du sujet, cadrage, choix technologiques, rédaction du cahier des charges.
Semaine 2	Conception : MCD / MLD / MPD, diagrammes UML, maquettes UI.
Semaine 3	Initialisation (Git, CI/CD), base de données, authentification JWT, back-office.
Semaine 4	Développement des fonctionnalités métier et du module IA.
Semaine 5	Application web / mobile, tableaux de bord, tests.
Semaine 6	Tests de sécurité, optimisation, documentation, préparation de la soutenance.

III. Conception

A. Diagramme de contexte

Le diagramme de contexte décrit les interactions globales entre le système MarketCraft et ses acteurs externes, sans détailler le fonctionnement interne.

Acteurs identifiés :

- Acheteur (visiteur ou utilisateur connecté).
- Vendeur (artisan gérant une boutique).
- Administrateur (modérateur de la plateforme).
- Serveur de base de données (MySQL).
- API externes : Claude / Anthropic (module IA), service d'envoi d'e-mails (SMTP / PHPMailer).

B. Diagramme de packages

Le diagramme de packages regroupe les composants backend selon leur fonction et reflète l'organisation MVC du projet.

- **Controllers** : AuthController, ProductController, BoutiqueController, OrderController, AvisController, SearchController, DashboardController, UploadController.
- **Models** : User, Boutique, Product, Order, Payment, Avis.
- **Core** : Router, Controller (base), Auth (JWT).
- **Routes** : web.php (déclaration de toutes les routes API).
- **Public** : index.php (front controller, point d'entrée unique).

C. Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation identifie les acteurs (acheteur, vendeur, administrateur, système, service de paiement) et l'ensemble de leurs interactions avec le système, avec les relations « include » (obligatoire) et « extend » (optionnelle). Les acteurs et leurs cas principaux :

- **Visiteur** : s'inscrire, se connecter, parcourir le catalogue, rechercher, consulter une fiche/boutique.
- **Acheteur** : gérer son panier, commander, payer, suivre ses commandes, déposer un avis vérifié.
- **Vendeur** : créer sa boutique, gérer ses produits et ses commandes, répondre aux avis.
- **Administrateur** : valider les boutiques, gérer les utilisateurs, arbitrer les litiges, modérer les avis.

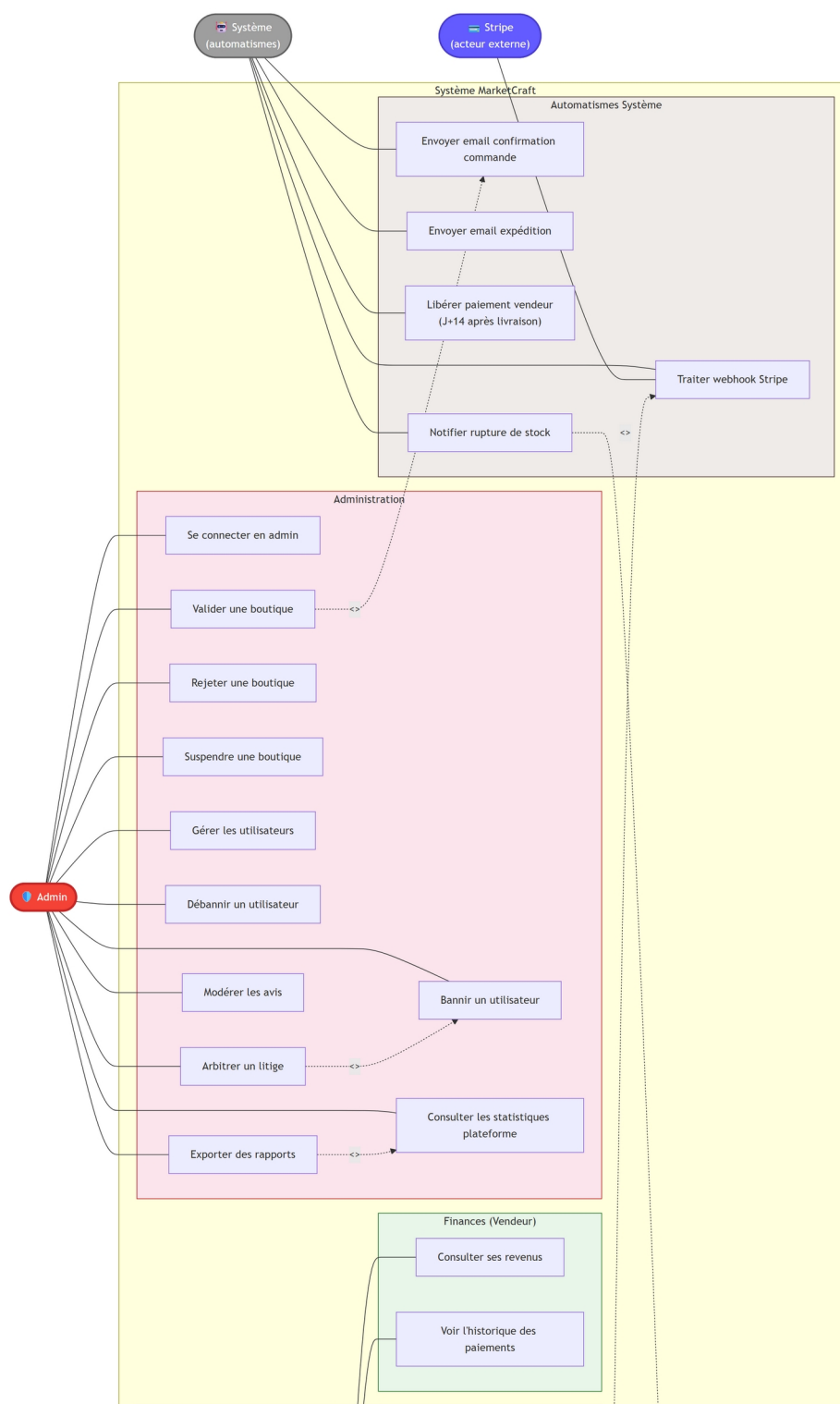


Figure 1 — Cas d'utilisation — authentification, catalogue et compte acheteur (vue 1/3)

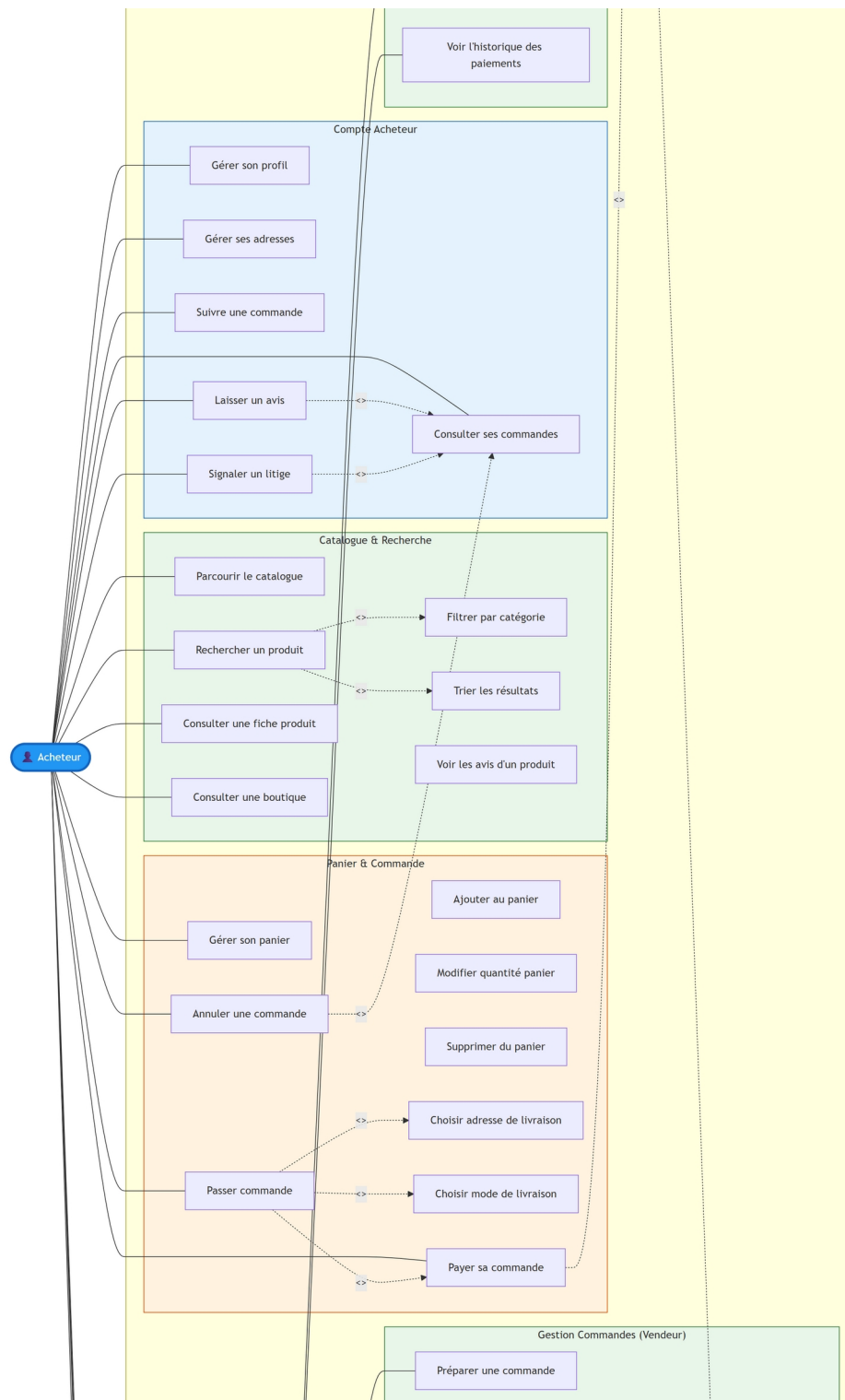


Figure 2 — Cas d'utilisation — panier, commande et gestion vendeur (vue 2/3)

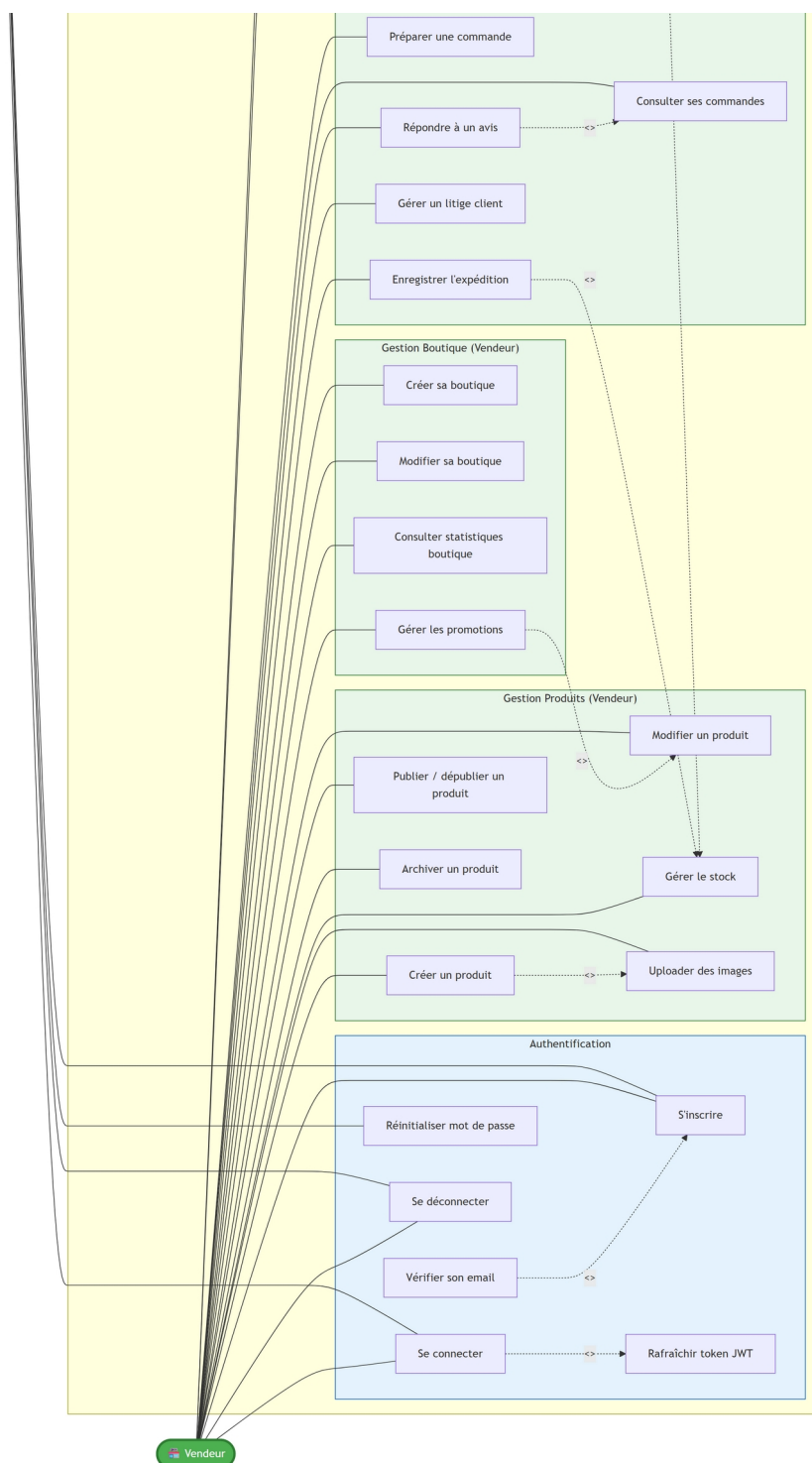


Figure 3 — Cas d'utilisation — administration et automatismes système (vue 3/3)

D. Scénarios

Trois scénarios fonctionnels clés illustrent les interactions entre l'utilisateur et le système.

1. Rechercher un produit avec l'IA et commander

Acteur principal : Acheteur.

Objectif : trouver un produit pertinent et le commander.

Déroulement :

1. L'acheteur saisit sa recherche en langage naturel dans la barre de recherche IA.
2. Le système envoie la requête à l'API Claude, qui retourne des mots-clés et filtres au format JSON.
3. Le backend interroge le catalogue par similarité et renvoie les produits pertinents en stock.
4. L'acheteur ajoute un produit au panier, puis valide la commande (adresse, mode de livraison).
5. Le système enregistre la commande et son paiement, puis affiche la confirmation.

Le diagramme de séquence ci-après détaille le parcours d'achat complet, de la connexion à la confirmation de commande.



2. Créer une boutique et publier un produit (vendeur)

Acteur principal : Vendeur connecté.

Déroulement :

1. Le vendeur crée sa boutique (nom, description, logo) ; elle passe en statut EN_ATTENTE.
2. Après validation par un administrateur, la boutique devient ACTIVE.
3. Le vendeur crée un produit : titre, description, prix, stock, catégorie, tags et images.
4. Le système enregistre le produit et le rend visible dans le catalogue (statut PUBLIE).

Le diagramme de séquence ci-après détaille le workflow vendeur, de la création de boutique au traitement des commandes.

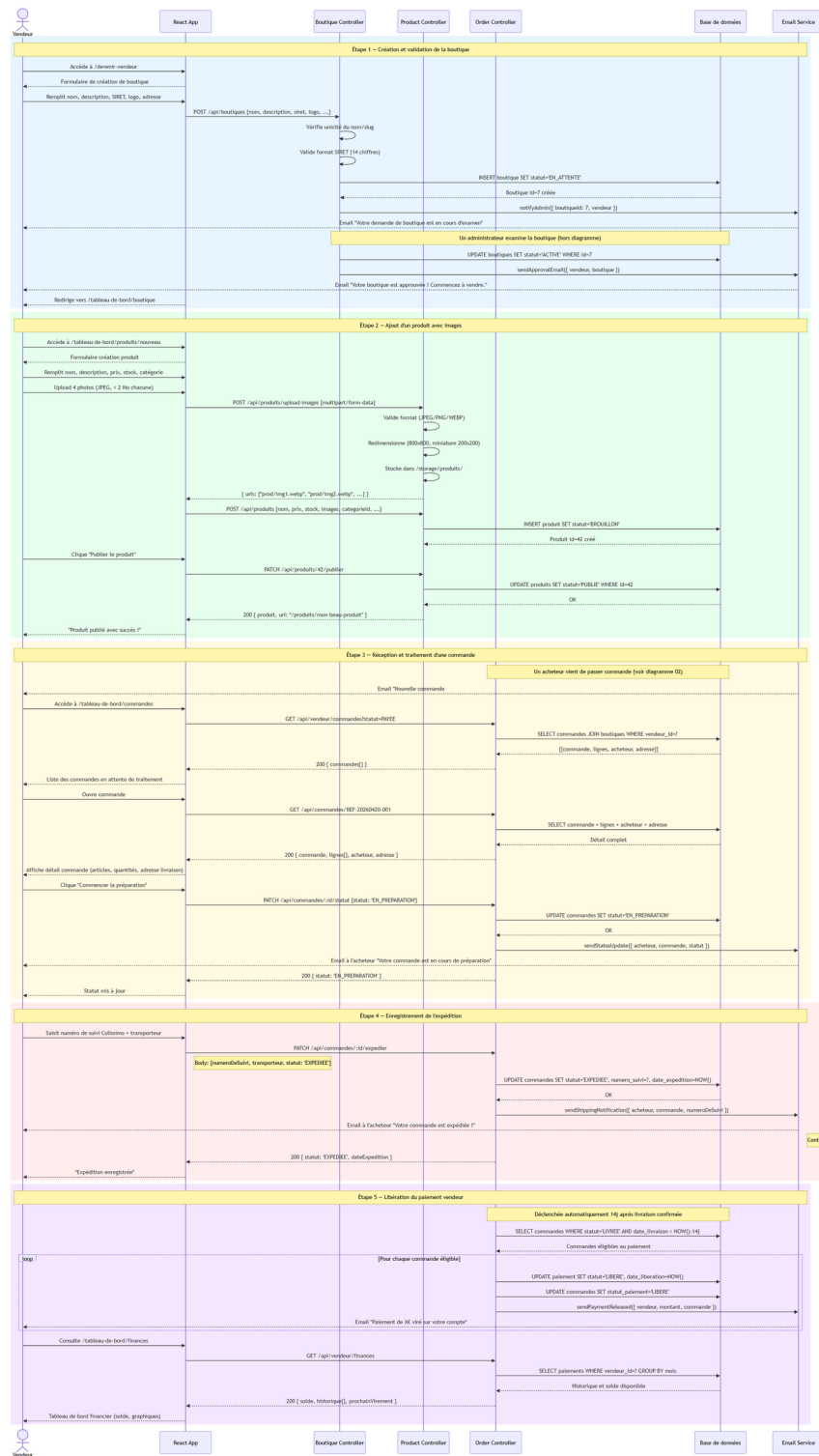


Figure 5 — Diagramme de séquence — workflow vendeur

3. Déposer un avis vérifié

Acteur principal : Acheteur ayant reçu sa commande.

Déroulement :

1. Depuis l'historique de commande, l'acheteur ouvre la fiche du produit reçu.
2. Il attribue une note (1 à 5) et rédige un commentaire.
3. Le système vérifie l'achat, enregistre l'avis et recalcule la note moyenne du produit.

E. Modèle conceptuel des données (MCD)

Le MCD représente les entités du domaine et leurs associations, indépendamment de toute technologie. Il met en évidence les règles de gestion : un acheteur ne peut pas créer de boutique ; un avis n'est posté que si la commande du produit est livrée ; une commande correspond à exactement un paiement.

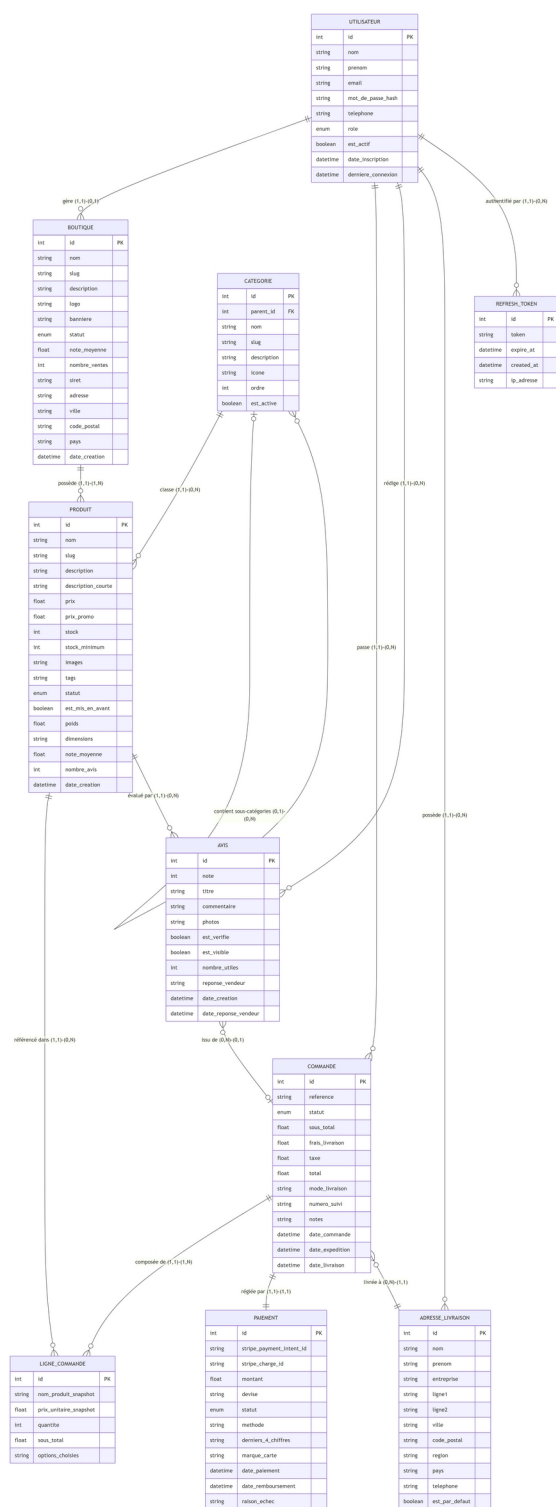


Figure 6 — Modèle Conceptuel de Données (MCD) — notation Merise

F. Diagramme de classes

Le diagramme de classes traduit le domaine métier en classes orientées objet, directement liées aux modèles PHP du backend. Chaque classe expose ses attributs typés et ses méthodes, ainsi que les multiplicités et compositions entre entités.

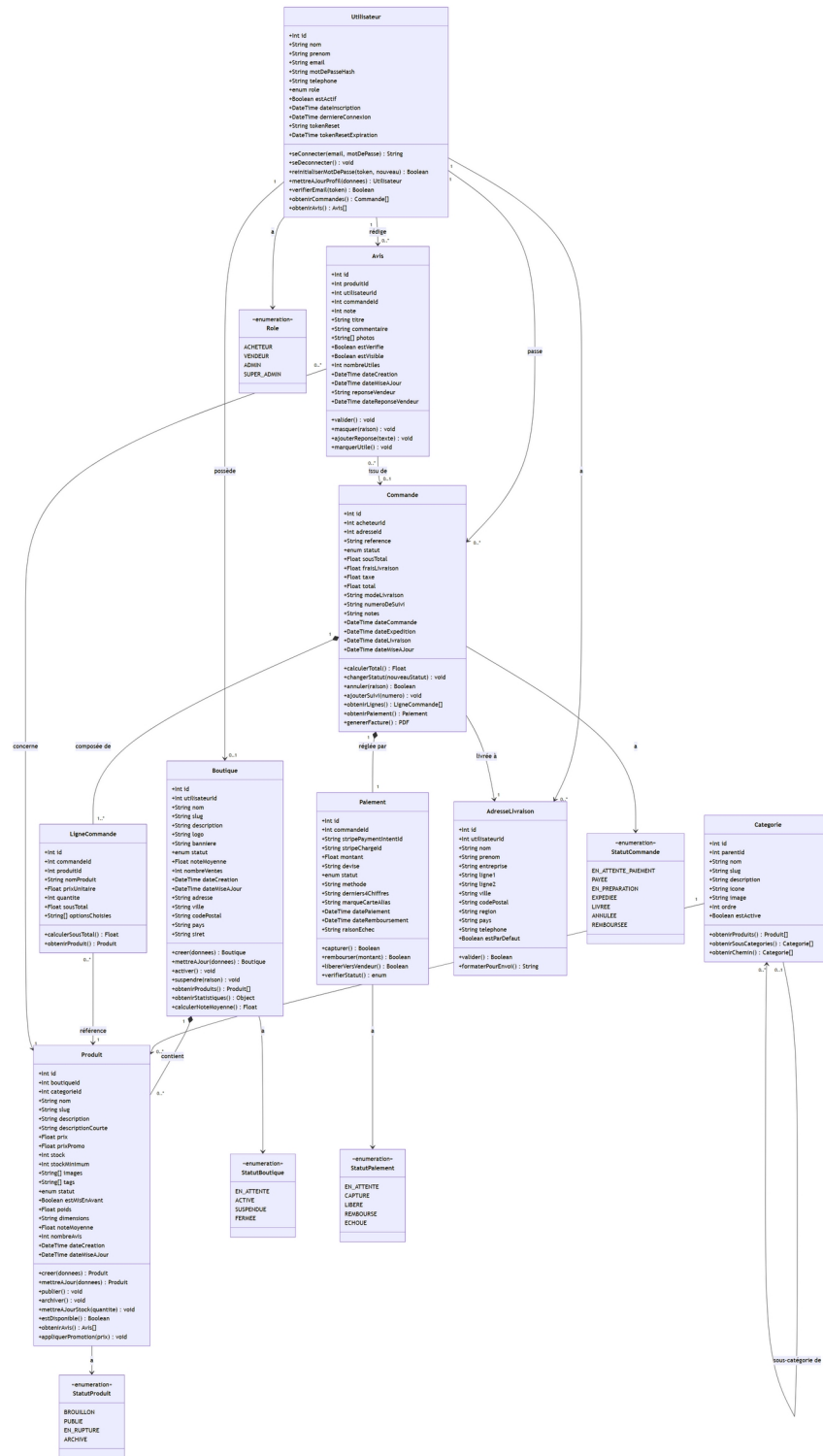


Figure 7 — Diagramme de classes UML du domaine métier

G. Modèle physique de données (MPD)

Le MPD correspond à la structure réelle de la base MySQL : 9 tables InnoDB en utf8mb4, avec clés primaires, clés étrangères, contraintes, index et un index FULLTEXT sur les produits. Le script de création complet figure en annexe et dans le fichier migrations.sql du dépôt.

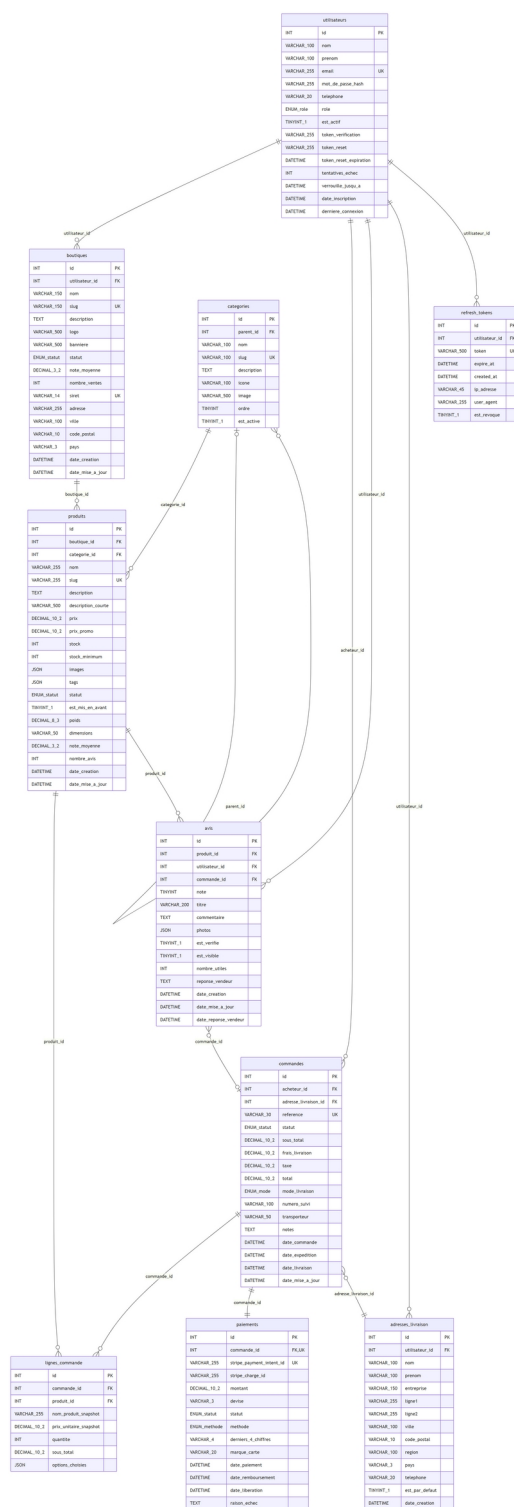


Figure 8 — Modèle Logique / Physique de Données (MLD) — tables et relations

H. Diagrammes d'activité

Les diagrammes d'activité modélisent les parcours utilisateur de bout en bout, avec leurs décisions, branchements et boucles. Trois parcours sont représentés : acheteur, vendeur et administrateur.

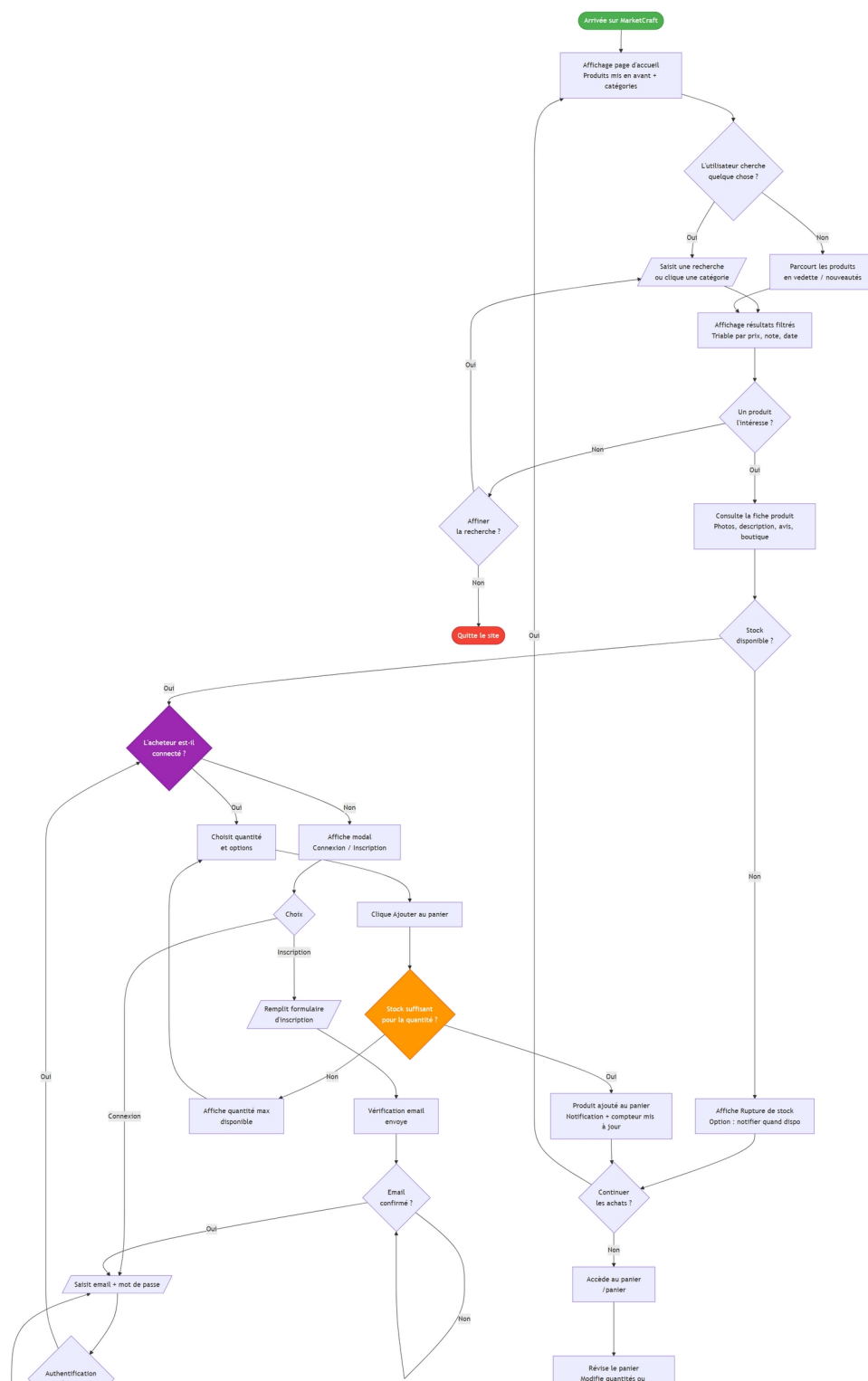


Figure 9 — Diagramme d'activité — parcours acheteur (1/2 : découverte et panier)

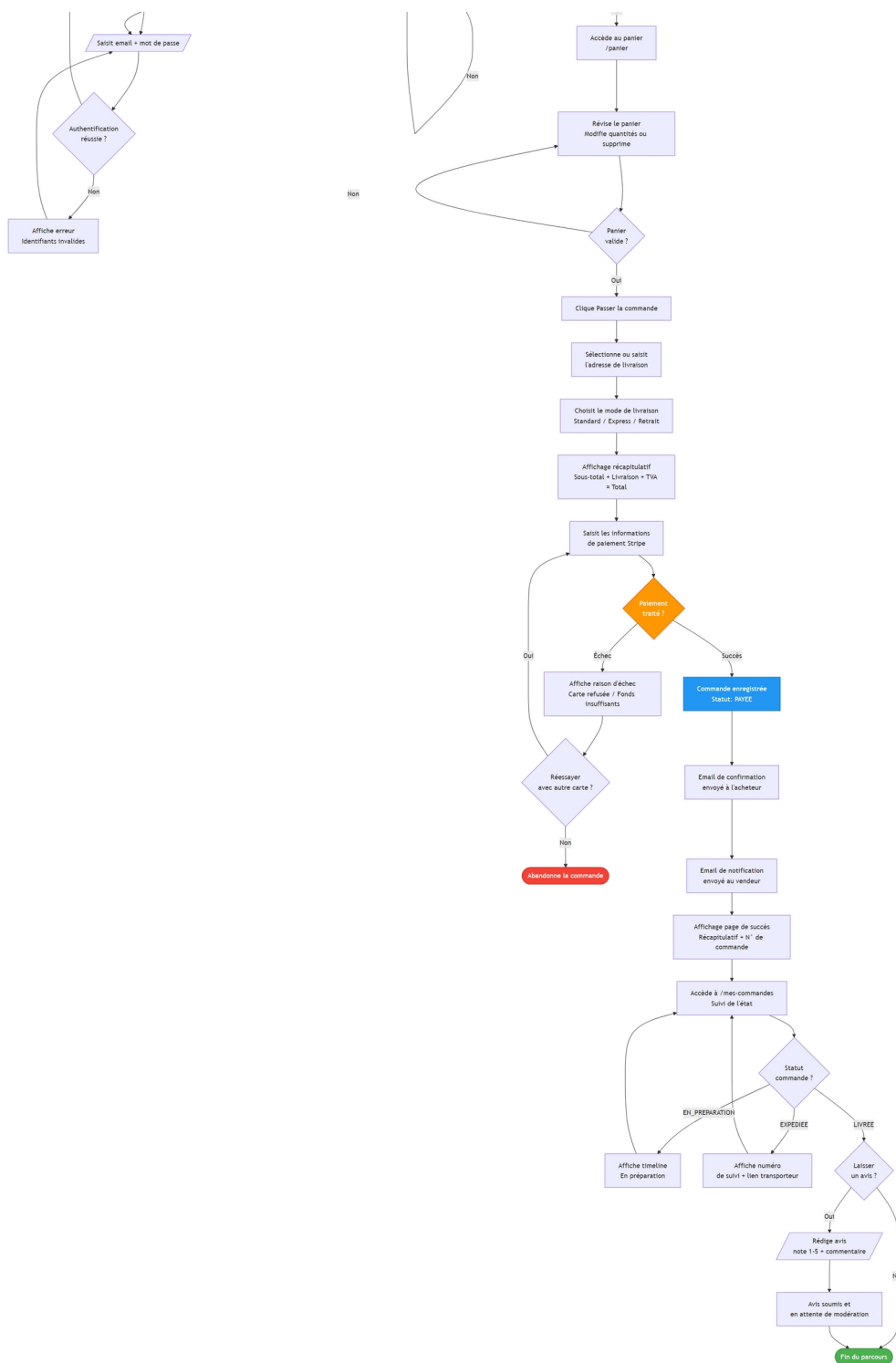


Figure 10 — Diagramme d'activité — parcours acheteur (2/2 : commande, paiement et suivi)



Le parcours d'administration (validation des boutiques, gestion des utilisateurs, arbitrage des litiges et modération) est représenté ci-dessous.

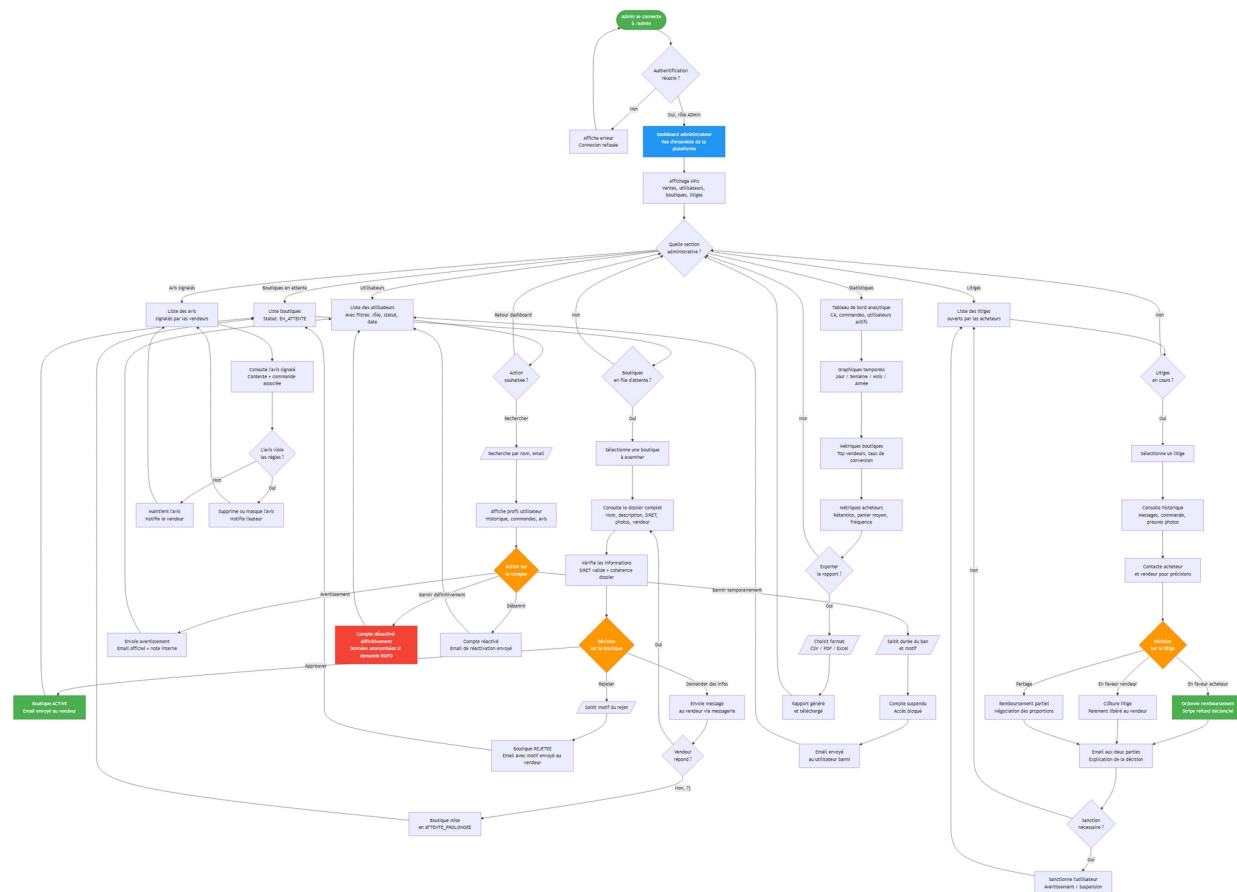


Figure 12 — Diagramme d'activité — parcours administrateur

IV. Tests

A. Objectifs des tests

Les tests visent à garantir la stabilité, la fiabilité et la sécurité de MarketCraft, et à vérifier que chaque fonctionnalité respecte le cahier des charges, y compris en cas d'erreurs ou de saisies invalides. Les objectifs principaux sont :

- Valider les flux d'inscription, de connexion et de rafraîchissement du token.
- Vérifier la création de boutiques et de produits, et le passage de commande.
- Tester la recherche IA et son mécanisme de repli.
- S'assurer de la protection des routes via JWT et du contrôle des rôles.

B. Types de tests réalisés

Les tests ont été réalisés manuellement et automatiquement sur les différentes couches de l'application :

- **Tests unitaires** : fonctions critiques (hachage de mot de passe, validation, routeur). Backend : PHPUnit (AuthTest, PasswordTest, RouterTest).
- **Tests frontend** : logique d'API, authentification et panier (Jest / React Testing Library).
- **Tests mobile** : service API de l'application React Native (Jest).
- **Tests d'API** : requêtes GET / POST / PUT / DELETE avec et sans token (Postman).
- **Tests de sécurité** : accès aux routes protégées sans token, injection SQL, brute-force.

C. Exemple de scénarios de test

Quelques scénarios concrets vérifiés durant le développement :

#	Scénario	Résultat attendu
1	Connexion avec un mauvais mot de passe	Erreur 401 (Unauthorized).
2	Accès à /orders sans token JWT	Erreur 401, accès refusé.
3	Création de produit par un acheteur	Erreur 403 (rôle insuffisant).
4	Recherche IA avec clé API absente	Bascule sur le fallback mots-clés, résultats renvoyés.
5	Dépôt d'un avis sans achat livré	Refus : avis non autorisé.
6	Commande d'un produit en rupture de stock	Refus : stock insuffisant.

D. Limites et améliorations possibles

La couverture des tests est satisfaisante mais perfectible :

- Étendre la couverture des tests automatisés (intégration de bout en bout).
- Renforcer l'intégration continue (GitHub Actions) pour tester à chaque push.
- Ajouter des tests de charge pour anticiper une forte volumétrie de produits et de commandes.

- Mettre en place des tests de non-régression sur le module IA (jeux de requêtes de référence).

V. Développement

A. Technologies choisies

Le choix des technologies s'est appuyé sur nos compétences, les objectifs du projet, la simplicité de mise en œuvre et la portabilité multi-plateforme :

- **Frontend web** : React 18 + Tailwind CSS, React Router, Context API.
- **Application mobile** : React Native 0.74 (TypeScript) + React Navigation.
- **Backend** : PHP 8.1, architecture MVC maison, API REST.
- **Base de données** : MySQL 8 (accès par PDO, requêtes préparées).
- **Authentification** : JSON Web Token (firebase/php-jwt).
- **Intelligence Artificielle** : API Claude (Anthropic).
- **E-mails** : PHPMailer (SMTP).
- **Outils** : Git / GitHub, Postman, Visual Studio Code, Composer, npm.

B. Environnement de travail

Le projet a été développé sous Windows, avec les outils suivants :

- IDE principal : Visual Studio Code (backend, frontend et mobile).
- Serveur local PHP (php -S) et base MySQL 8.
- Gestion des dépendances : Composer (PHP) et npm (JS/TS).
- Postman pour tester les routes de l'API ; Git et GitHub pour la gestion de versions.

C. Architecture MVC / Installation

Le backend suit une architecture MVC personnalisée. Toutes les requêtes passent par un point d'entrée unique (public/index.php) ; le routeur associe chaque route à une méthode de contrôleur, qui sollicite les modèles et renvoie une réponse JSON. Les middlewares (notamment l'authentification JWT) s'exécutent avant le contrôleur.

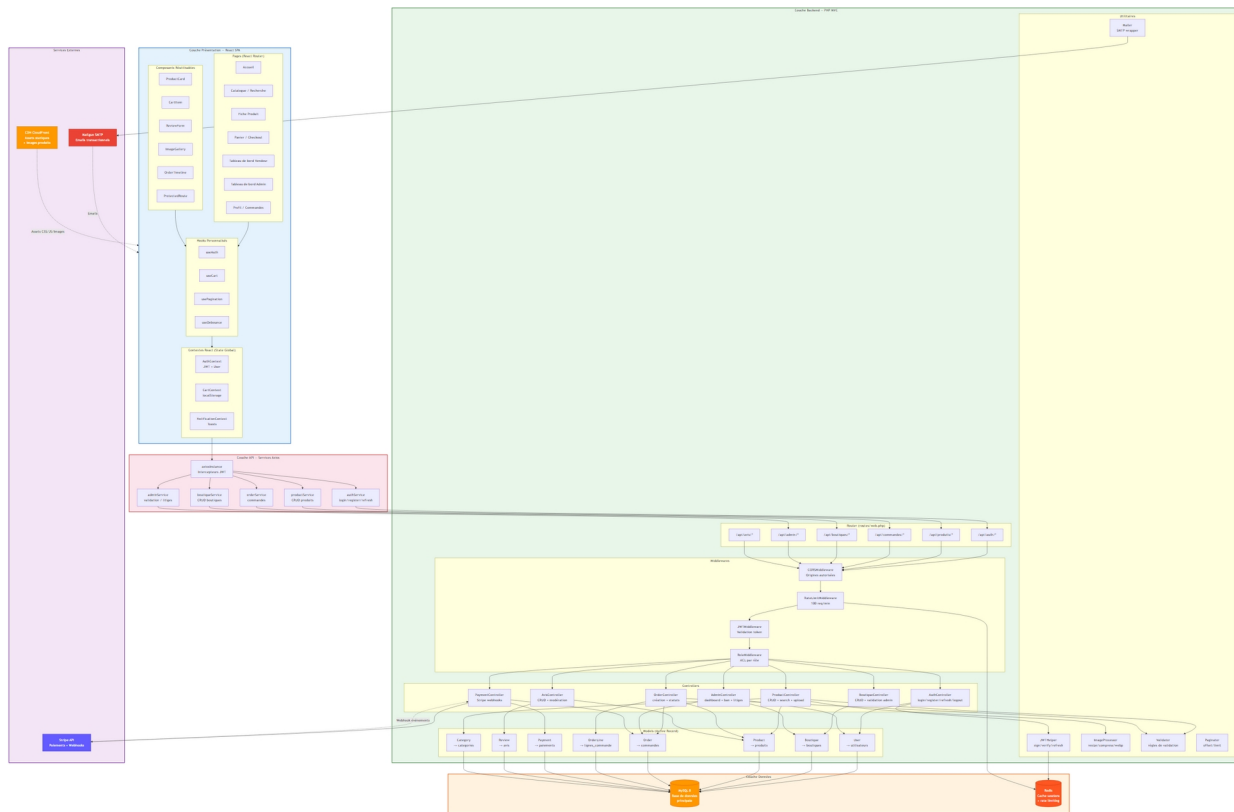


Figure 13 — Architecture applicative en couches (frontend, API, données, services)

Organisation des dossiers :

- **app/Controllers/** — logique des actions (Auth, Product, Boutique, Order, Avis, Search...).
- **app/Models/** — entités liées aux tables (User, Boutique, Product, Order, Payment, Avis).
- **app/Core/** — classes utilitaires (Router, Controller, Auth).
- **routes/web.php** — déclaration des routes et de leurs middlewares.
- **public/index.php** — front controller, reçoit toutes les requêtes API.

Installation locale :

1. Cloner le dépôt depuis GitHub.
2. Backend : composer install, copier .env.example en .env et configurer la base + JWT_SECRET.
3. Importer le schéma : `mysql -u root -p < migrations.sql`.
4. Lancer l'API : `php -S localhost:8000 -t public`, puis le frontend : `npm install && npm start`.

D. Backend

1. Modèles

Les modèles représentent les entités principales et encapsulent l'accès aux tables via PDO :

- **User** : utilisateur (id, email, mot_de_passe_hash, role, est_actif...).
- **Boutique** : boutique d'un vendeur (nom, slug, statut, note_moyenne).
- **Product** : produit (nom, prix, stock, tags, statut), rattaché à une boutique.
- **Order / Payment** : commande, lignes et paiement associé.

- **Avis** : note et commentaire vérifiés d'un acheteur.

2. Routes / API

Les routes sont déclarées dans routes/web.php. Chaque route est associée à une méthode de contrôleur, avec d'éventuels middlewares. Aperçu des principales routes :

Méthode	Route	Action	Auth
POST	/auth/register	Inscription	—
POST	/auth/login	Connexion (JWT)	—
POST	/auth/refresh	Rafraîchir le token	—
GET	/products	Liste + filtres	—
POST	/products	Créer un produit	auth
POST	/boutiques	Créer une boutique	auth
POST	/orders	Passer une commande	auth
POST	/products/:id/avis	Déposer un avis	auth
POST	/search/ai	Recherche IA	—
GET	/dashboard/stats	Statistiques vendeur	auth

3. Contrôleurs

Chaque contrôleur gère une entité et orchestre la logique métier : il valide les données d'entrée, sollicite les modèles, et renvoie une réponse JSON avec le code HTTP approprié.

- **AuthController** : inscription, connexion, refresh, profil.
- **ProductController / BoutiqueController** : CRUD produits et boutiques.
- **OrderController** : création de commande et mise à jour de statut.
- **AvisController** : dépôt et modération des avis.
- **SearchController** : recherche IA (Claude) et recherche mots-clés.
- **DashboardController / UploadController** : statistiques et upload d'images.

4. Sécurité / Authentification

L'authentification repose sur les JSON Web Tokens (JWT). À la connexion, un token signé (HS256) est généré et renvoyé au client ; il doit ensuite être présenté dans l'en-tête Authorization: Bearer pour accéder aux routes protégées. Mesures de sécurité mises en place :

- Hachage des mots de passe (bcrypt) — jamais stockés en clair.
- Middleware JWT (Core/Auth) validant le token sur les routes sensibles, avec gestion de l'expiration.
- Refresh token (30 jours) pour renouveler la session sans reconnexion.
- Contrôle des rôles et de la propriété des ressources (isOwnerOrAdmin).
- Requêtes préparées (PDO) pour prévenir les injections SQL.
- Verrouillage du compte après plusieurs tentatives échouées (tentatives_echec, verrouille_jusqu_a).

- Secrets (clé JWT, identifiants BDD, clé API IA) externalisés dans le fichier .env, jamais commité.

Le diagramme de séquence ci-après illustre le cycle de vie complet de l'authentification : inscription, connexion et génération du JWT, requête protégée, rafraîchissement du token et déconnexion.

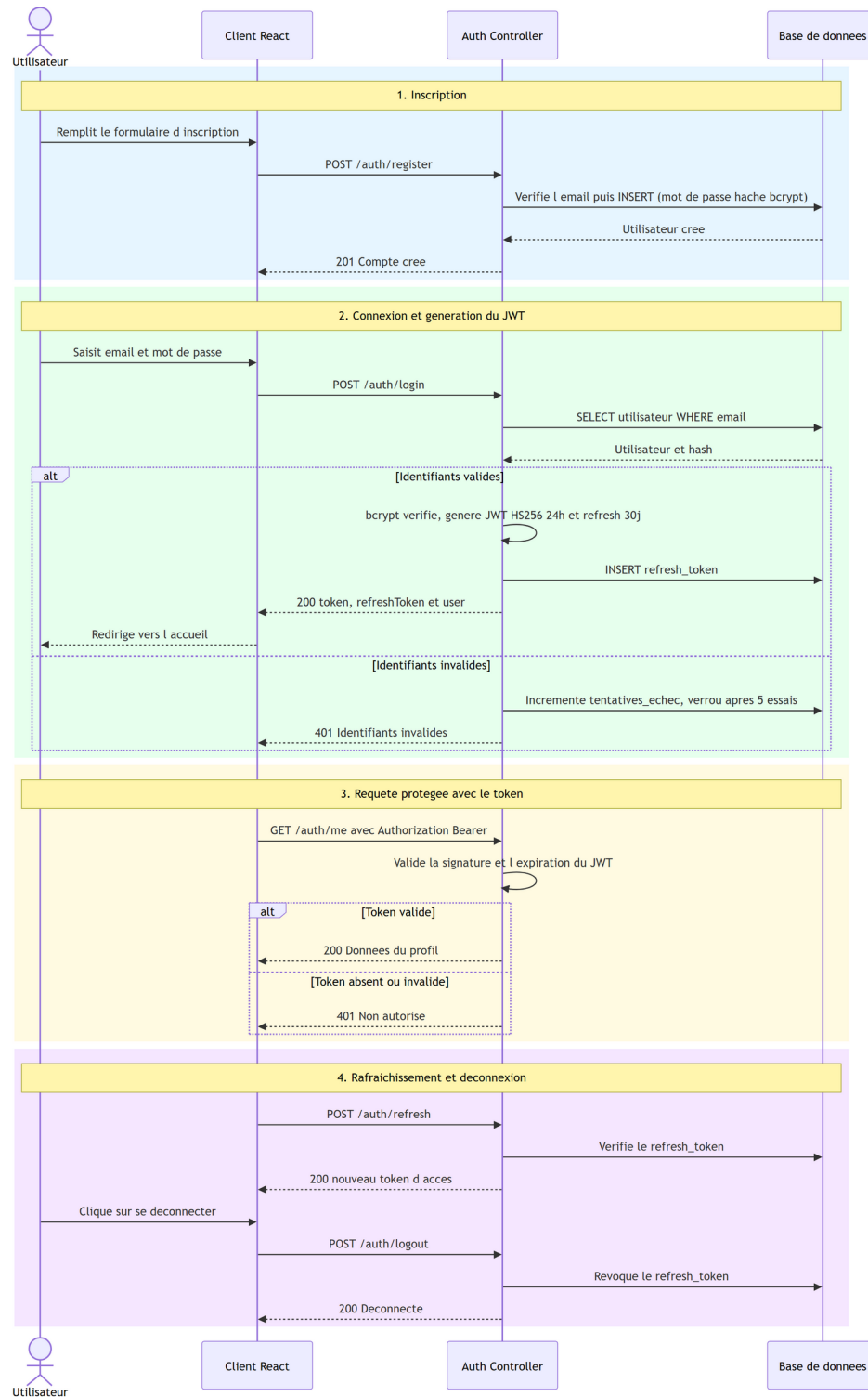


Figure 14 — Diagramme de séquence — authentification JWT (inscription, connexion, refresh, logout)

E. Frontend (React)

Le frontend est développé en React 18 selon une architecture modulaire par composants, avec Tailwind CSS pour un rendu responsive. La navigation est gérée par React Router ; l'état global (authentification, panier) repose sur le Context API.

Pages et composants principaux :

- HomePage, ProductsPage, ProductDetailPage : catalogue et fiches produits.
- AISearchBar / SearchResultsPage : recherche en langage naturel et affichage des recommandations.
- CartPage / CheckoutPage : panier persistant (localStorage) et tunnel d'achat.
- DashboardPage / BuyerStatsPage : tableaux de bord vendeur et acheteur.
- LoginPage / RegisterPage / ProfilePage : authentification et compte.

Les appels à l'API passent par une instance Axios centralisée (src/services/api.js) qui injecte automatiquement le token JWT dans les en-têtes et gère son rafraîchissement. Les maquettes de l'interface figurent en Annexe E.

F. Application mobile (React Native)

L'application mobile, développée en React Native 0.74 (TypeScript), offre les mêmes fonctionnalités que le web grâce au partage de la logique. La navigation combine une barre d'onglets et une pile d'écrans :

- Onglets : Accueil, Catalogue, Panier (avec badge), et selon le rôle : Dashboard (vendeur) ou Mes stats (acheteur).
- Écrans : détail produit, commande (checkout), connexion / inscription.
- Persistance de session via AsyncStorage ; appels API mutualisés avec le web (Axios).

G. Module IA — recherche et recommandation

Le module IA est exposé par la route POST /search/ai et implémenté dans SearchController. Son fonctionnement :

1. La requête en langage naturel de l'acheteur est envoyée à l'API Claude avec un prompt système strict.
2. Claude répond uniquement en JSON : mots-clés, catégories, prix_min, prix_max et un message convivial.
3. Le backend nettoie la réponse, valide le JSON, puis construit une requête SQL préparée (LIKE sur nom / description / tags + filtres de prix), en ne retenant que les produits en stock.
4. En cas d'indisponibilité de l'IA (clé absente, timeout 15 s, JSON invalide), un fallback local tokenise la requête et recherche par mots-clés.

Sécurité du module : la sortie de l'IA n'est jamais injectée directement dans le SQL ; elle alimente des paramètres liés (requêtes préparées). La clé API est stockée dans le fichier .env.

H. Déploiement

L'architecture cible de production organise l'infrastructure en zones de sécurité (Internet, DMZ, backend, base de données isolée), avec un reverse proxy Nginx (HTTPS), PHP-FPM, un cache Redis, une base MySQL (primaire + réplica) et des services tiers (paiement, e-mails, stockage objet).

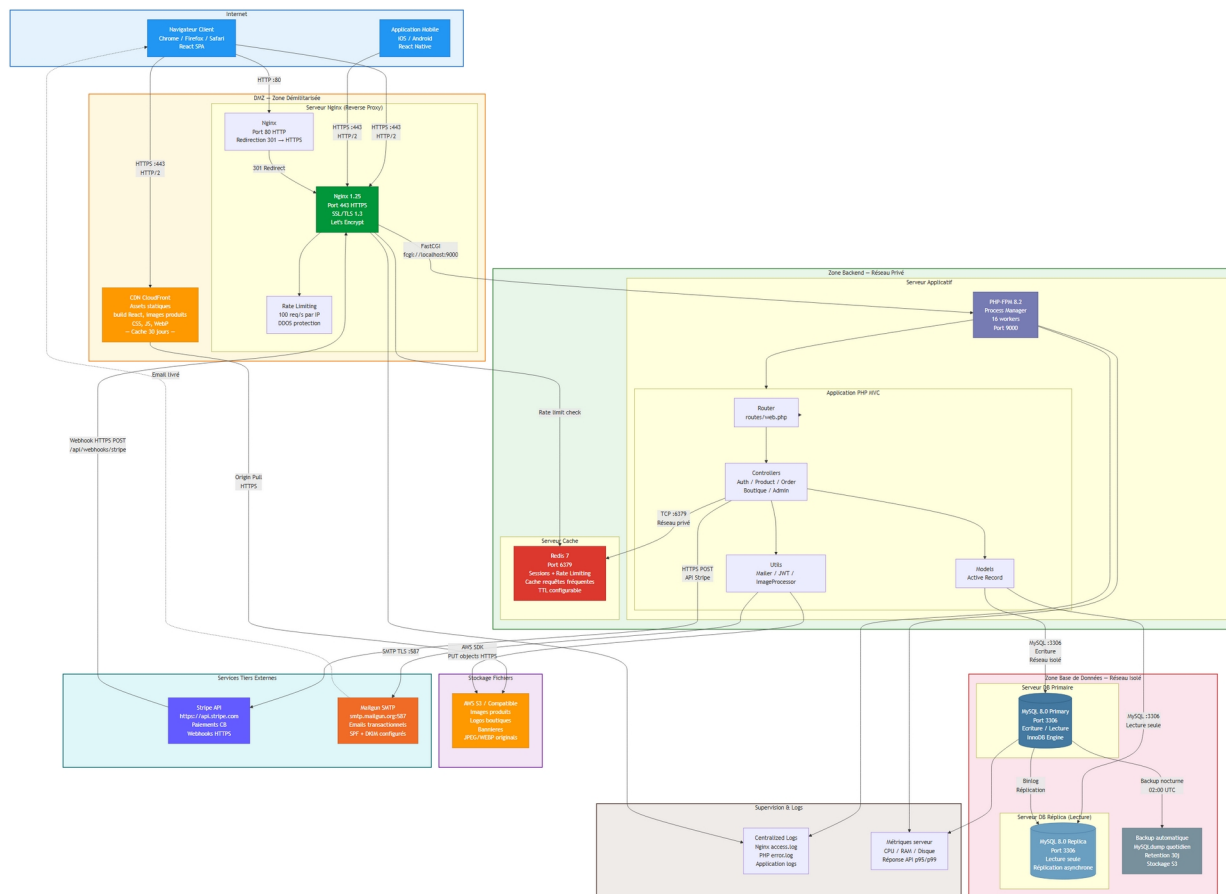


Figure 15 — Diagramme de déploiement — infrastructure de production

VI. Perspectives

Dans une optique d'amélioration continue, plusieurs pistes peuvent enrichir MarketCraft et renforcer sa valeur, tant pour les acheteurs que pour les artisans.

Paiement en ligne réel. Intégrer une passerelle de paiement (Stripe) avec gestion des webhooks et libération différée des fonds au vendeur, afin de sécuriser les transactions de bout en bout.

Recommandations enrichies. Étendre le module IA à des recommandations basées sur l'historique de navigation et le contenu du panier (« vous pourriez aussi aimer... »), au-delà de la seule recherche.

Messagerie acheteur-vendeur. Permettre des échanges directs (questions sur un produit, suivi de commande) au sein de la plateforme.

Notifications et e-mails transactionnels. Automatiser les e-mails de confirmation de commande et d'expédition, ainsi que des notifications push sur mobile.

Tableau de bord administrateur avancé. Statistiques globales de la plateforme, exports de rapports et outils de modération renforcés.

VII. Conclusion

Le projet MarketCraft a été conçu avec l'ambition de valoriser l'artisanat en ligne et de simplifier la découverte de produits faits main. En partant d'un constat concret — la difficulté pour les artisans de vendre et pour les acheteurs de trouver le bon produit — le projet propose une réponse technique claire, structurée et accessible. Il combine une interface moderne en React, une application mobile React Native, un backend solide en PHP avec architecture MVC, une base de données bien modélisée, et un module d'Intelligence Artificielle apportant une réelle valeur ajoutée.

Sur le plan technique, ce projet m'a permis de renforcer mes compétences full-stack : modélisation de données, conception d'une API REST sécurisée avec authentification JWT, intégration d'un service d'IA, développement web et mobile, et application des bonnes pratiques de sécurité et de versioning. Le travail en binôme, conduit en mode agile, a également développé mes compétences d'organisation et de collaboration.

En résumé, MarketCraft est un projet abouti et cohérent, dont la base technique solide ouvre de nombreuses possibilités d'évolution. Les perspectives évoquées montrent qu'il peut devenir une véritable marketplace de référence pour l'artisanat, connectée à l'écosystème numérique moderne.

VIII. Webographie

- Documentation officielle React : <https://react.dev/>
- Documentation React Native : <https://reactnative.dev/>
- PHP – Documentation : <https://www.php.net/manual/fr/>
- JWT.io – JSON Web Tokens : <https://jwt.io/>
- firebase/php-jwt : <https://github.com/firebase/php-jwt>
- Tailwind CSS : <https://tailwindcss.com/>

- Documentation API Claude (Anthropic) : <https://docs.anthropic.com/>
- PHPMailer : <https://github.com/PHPMailer/PHPMailer>
- MySQL – Documentation : <https://dev.mysql.com/doc/>

IX. Annexes

Annexe A — Structure de la base de données

```
utilisateurs : id, nom, prenom, email (UK), mot_de_passe_hash, role, est_actif, ...
boutiques : id, utilisateur_id (FK), nom, slug (UK), statut, note_moyenne, siret
categories : id, parent_id (FK), nom, slug (UK), est_active
produits : id, boutique_id (FK), categorie_id (FK), nom, prix, stock, tags (JSON), statut
commandes : id, acheteur_id (FK), adresse_livraison_id (FK), reference (UK), statut, total
lignes_commande: id, commande_id (FK), produit_id (FK), quantite, prix_unitaire_snapshot
paiements : id, commande_id (FK/UK), montant, devise, statut, methode
avis : id, produit_id (FK), utilisateur_id (FK), commande_id (FK), note, est_verifie
```

Annexe B — Exemple de payload JWT

```
$payload = [
  'iss' => 'marketcraft',      // émetteur
  'sub' => '3',                // ID de l'utilisateur
  'email' => 'antoine@example.com',
  'role' => 'vendeur',         // rôle pour le contrôle d'accès
  'iat' => 1720723872,         // date d'émission (timestamp UNIX)
  'exp' => 1720810272         // date d'expiration
];
```

Annexe C — Exemple de requête API (Axios)

```
axios.post('/auth/login', {
  email: 'user@example.com',
  password: '*****'
}).then(res => {
  localStorage.setItem('token', res.data.token);
});
```

Annexe D — Module IA : réponse JSON

Réponse JSON normalisée renvoyée par Claude puis exploitée par le backend :

```
{
  "keywords": ["couverts", "dorés", "table", "bois"],
  "categories": ["mobilier"],
  "prix_min": null,
  "prix_max": 50,
  "message": "Je cherche des couverts dorés et une table en bois pour toi !"
}
```

Annexe E — Maquettes de l'interface

Cette annexe présente la charte graphique et les principales maquettes de l'interface web de MarketCraft (réalisées lors de la phase de conception). Elles définissent l'identité visuelle artisanale (tons terracotta et olive, typographie sérif) et la structure des écrans clés.

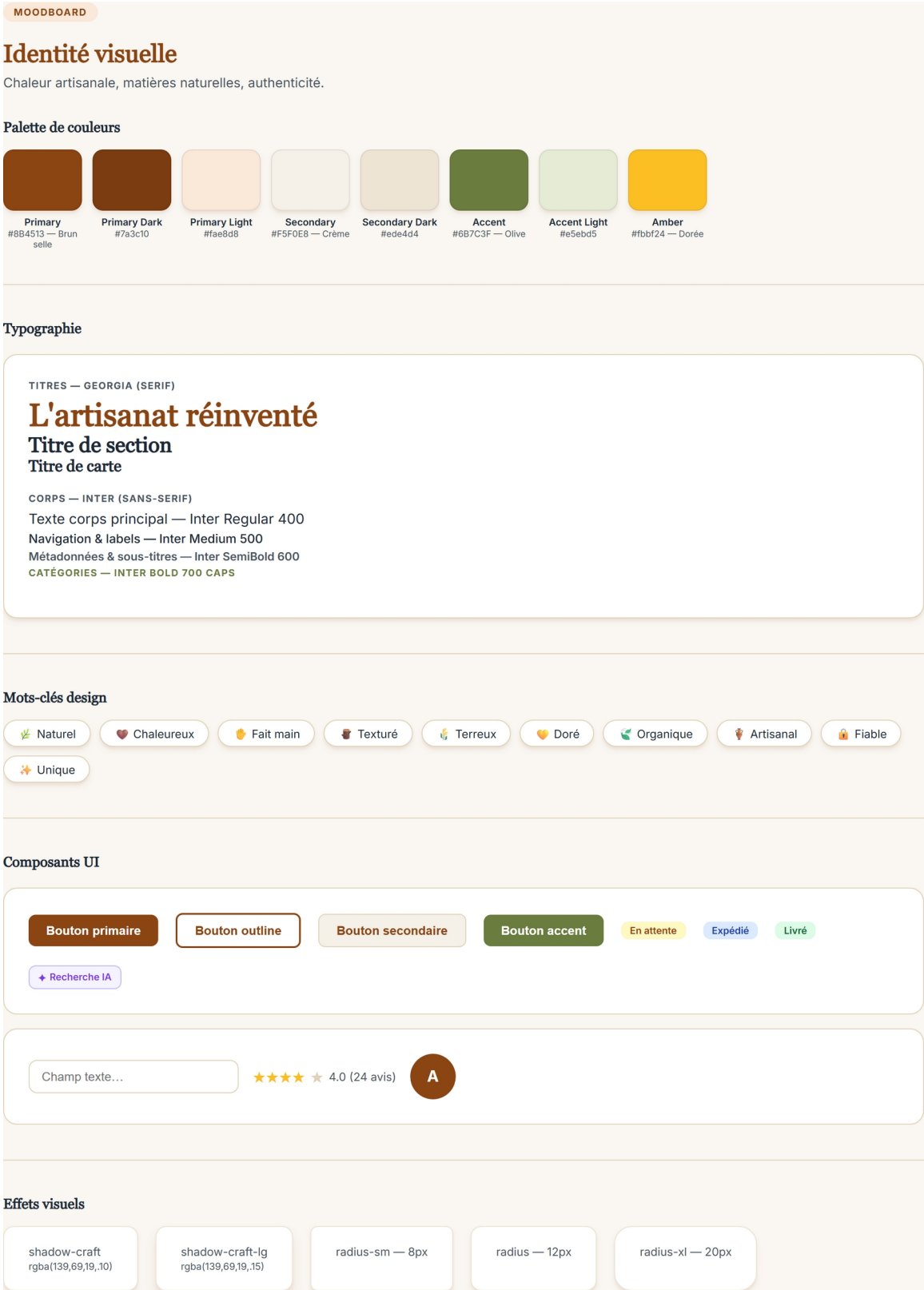


Figure 16 — Charte graphique — couleurs, typographie et composants



Figure 17 — Maquette — barre de navigation

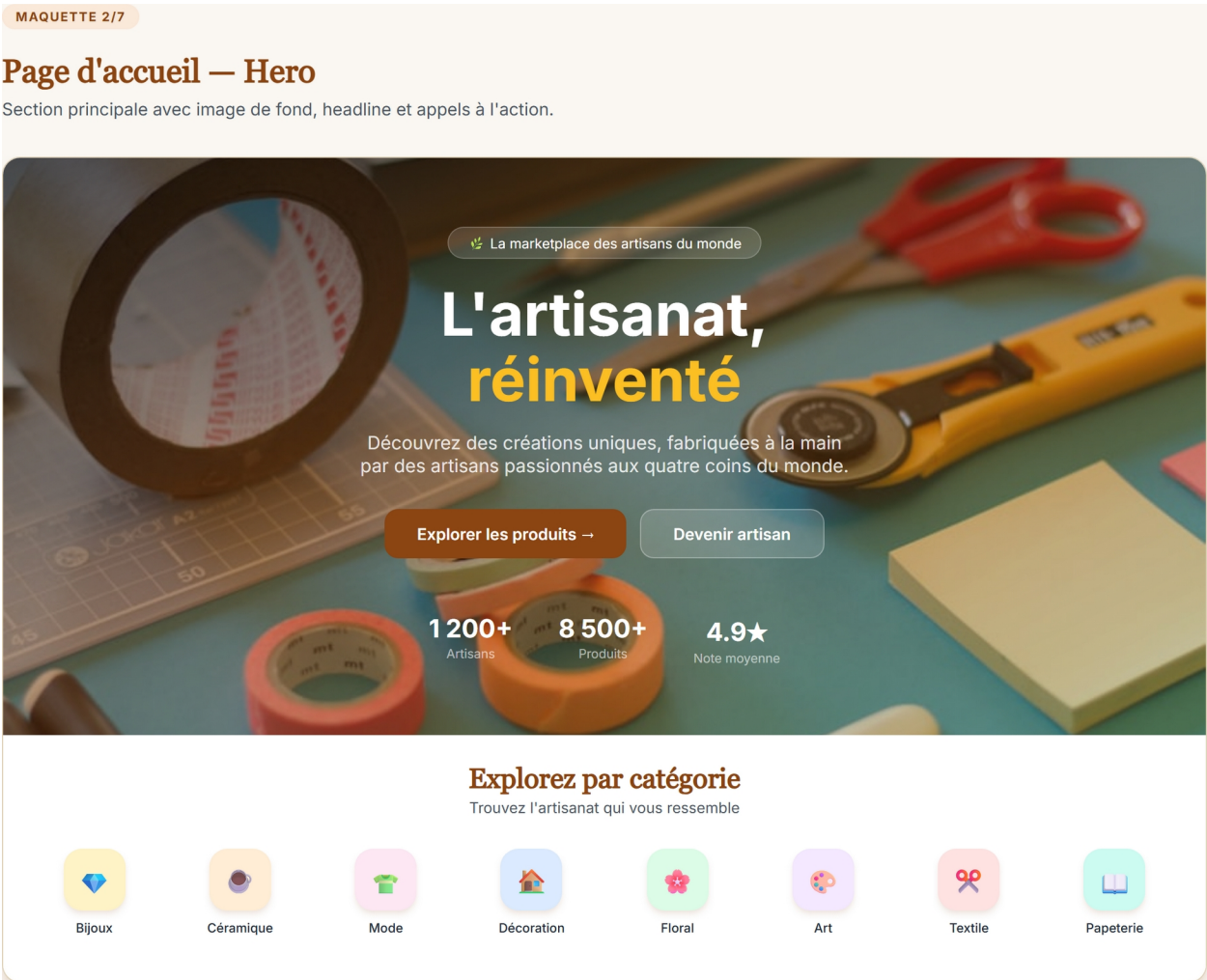


Figure 18 — Maquette — page d'accueil (hero et catégories)

MAQUETTE 3/7

Grille Produits

Affichage 4 colonnes — ProductCard avec image, note, prix et ajout au panier.

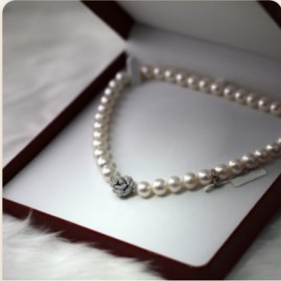
Produits tendance

Les coups de cœur de notre communauté

Voir tout →



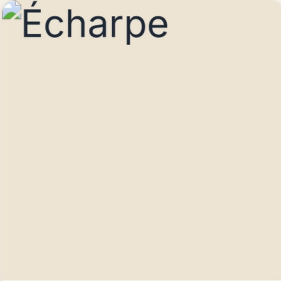
CÉRAMIQUE
Vase en céramique fait main
Céramiques de Lyon
★★★★★ 4.8 (24)
45 € + Panier



BIJOUX
Collier en argent ciselé
Bijoux Céleste
★★★★★ 4.9 (18)
89 € + Panier



BIEN-ÊTRE
Bougie parfumée cire d'abeille
Artisan du Midi
★★★★★ 4.6 (41)
18 € + Panier



Écharpe

TEXTILE
Écharpe en laine mérinos
Maison Textile
★★★★☆ 4.4 (12)
65 € + Panier

Figure 19 — Maquette — grille de produits

Antoine Brigouleix — Concepteur Développeur d'Applications · 2025–2026 · 41 / 45

MAQUETTE 4/7

Boutiques à la une

Cards boutique avec image en aspect 16/9, gradient overlay et description.

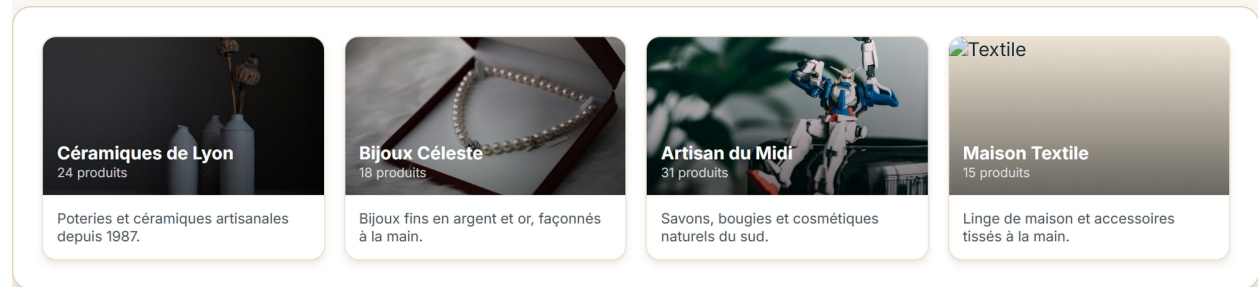


Figure 20 — Maquette — boutiques à la une

MAQUETTE 5/7

Authentification

Formulaires de connexion et d'inscription.

Connexion

Ravis de vous revoir sur MarketCraft

Adresse e-mail

Mot de passe

Se connecter

Pas encore de compte ? [S'inscrire](#)

Créer un compte

Rejoignez la communauté MarketCraft

Prénom

Nom

E-mail

Mot de passe

☒ Acheteur ☐ Vendeur

Créer mon compte

Déjà un compte ? [Connexion](#)

Figure 21 — Maquette — authentification (connexion / inscription)

MAQUETTE 6/7

Panier & Checkout

Tiroir panier latéral et page de confirmation de commande.

← Contenu de la page

 **Mon panier (3)**



Vase céramique
Qté : 1

45 €



Collier argent
Qté : 2

178 €



Bougie cire d'abeille
Qté : 1

18 €

Total

241 €

Commander →

Finaliser ma commande

Adresse de livraison

Prénom

Nom

Jean

Dupont

Adresse

12 rue de la Paix

Code postal

Ville

75001

Paris

Sous-total

Livraison

Total

241 €

Gratuite

241 €

Confirmer la commande ✓

Figure 22 — Maquette — panier et tunnel de commande

Dashboard Vendeur

Bonjour, Marie 🙌

3 240 €

47

4.8 ★

24

Commande	Client	Montant	Statut	Date
#1042	Sophie M.	90 €	En attente	04/06/2026
#1041	Paul D.	45 €	Expédié	02/06/2026
#1038	Laura B.	135 €	Livré	28/05/2026

Antoine Brigouleix — Concepteur Développeur d'Applications · 2025–2026 · 45 / 45